

Visual Exploratory Data Analysis (VisEDA): PART C

Hyperspectral Exploratory Data Analysis

A Guide to Hyperspectral and Multispectral Dataset Analysis

Isaac Osei Agyemang

Version 1.0.0

2026

Preface

VisEDA was created to bring the discipline and convenience of tabular exploratory data analysis to visual datasets. Hyperspectral data requires especially careful EDA because every sample contains both spatial structure and a dense spectral signature. A dataset can appear visually plausible while still containing defective bands, incompatible wavelength definitions, inconsistent radiometric scales, class imbalance, spectral redundancy, or corrupt files.

HyperspectralEDA addresses these issues through one unified interface for individual cubes and multi-cube datasets. It analyses spatial dimensions, per-band statistics, signal-to-noise ratio, robust noise, spectral smoothness, inter-band correlation, dropout bands, vegetation and water indices, principal components, texture, and cross-cube spectral diversity. The module also produces detailed dashboards, dedicated spectral plots, a self-contained HTML report, and programmatic access to all computed fields.

Isaac Osei Agyemang

May 2026

Table of Contents

Preface	2
Table of Contents	3
Chapter 1 — Introduction	7
1.1 What is HyperspectralEDA?	7
1.2 Why Hyperspectral EDA Matters	7
1.3 The Hyperspectral Cube	7
1.4 Design Philosophy	7
1.5 Installation	7
1.6 Supported Data Formats	8
1.7 Wavelength Conventions	8
Chapter 2 — Quick Start	9
2.1 Analyse a Single Cube	9
2.2 Analyse a Directory of Cubes.....	9
2.3 Load NumPy Arrays Directly.....	9
2.4 Get the Summary Dictionary.....	10
2.5 Generate All Visualisations.....	10
2.6 Extract a Pixel Spectral Signature	10
2.7 Compute Spectral Index Maps.....	10
2.8 Obtain PCA Scores	10
2.9 Generate the HTML Report	11
2.10 Command-Line Interface	11
Chapter 3 — The HyperspectralEDA Class	12
3.1 Constructor	12
3.2 load() — Load Cubes from Files	12
3.3 load_arrays() — Load In-Memory Cubes	12
3.4 summary() — Aggregate Statistics	12
3.5 get_record() — Access a CubeRecord	13
3.6 spectral_signature() — Read One Pixel Spectrum.....	13
3.7 compute_index() — Create an Index Map	13
3.8 pca_scores() — Obtain Pixel-Level Components	13
3.9 report() — Generate HTML	13
3.10 Object State and Method Chaining	13
Chapter 4 — The CubeRecord Object	14
4.1 Identity and Shape Fields.....	14
4.2 Global and Per-Band Fields	14
4.3 Spectral Quality and Spatial Fields	14
4.4 Spectral Indices, PCA, and Texture.....	14

Chapter 5 — What Is Analysed	16
5.1 Dataset Inventory	16
5.2 Spatial Dimensions and File Size	16
5.3 Global Radiometric Statistics	16
5.4 Per-Band Mean, Standard Deviation, Minimum, and Maximum	16
5.5 Signal-to-Noise Ratio	16
5.6 Robust Noise Using Median Absolute Deviation (<i>MAD</i>)	16
5.7 Saturation Fraction	16
5.8 Spectral Smoothness	17
5.9 Inter-Band Correlation	17
5.10 Dropout Band Detection	17
5.11 Spatial Mean and Spatial Standard-Deviation Maps	17
5.12 Spectral Indices	17
5.13 Principal Component Analysis	17
5.14 GLCM Texture on the First Principal Component	17
5.15 Cross-Cube Mean Spectrum	18
5.16 Spectral Diversity	18
5.17 Corrupt-File Detection	18
Chapter 6 — Visualisations	19
6.1 plot() — Single-Cube Deep-Dive Dashboard	19
Cube Overview	20
False Colour	20
Spatial Mean Map	20
Mean Spectral Signature $\pm 1\sigma$	20
SNR per Band	20
Per-Band Min / Mean / Max	20
NDVI, NDWI, and EVI maps	20
PCA Variance	20
Texture (GLCM)	20
Spectral Quality Metrics	20
6.2 plot_dataset() — Dataset Analysis Dashboard	20
6.3 plot_spectra() — Spectral Overlay	22
6.4 plot_false_colour() — False-Colour Previews	22
6.5 plot_ndvi() — NDVI Map Grid	23
6.6 plot_pca_components() — PCA Component Images	23
6.7 plot_band_stats() — Per-Band Diagnostics	24
6.8 plot_spectral_diversity() — Cosine Similarity	25
Chapter 7 — Interpreting Results and Common Findings	26
7.1 Red Flags to Review	26

7.2 Interpreting the Attached Example Outputs.....	26
7.3 Reflectance, Radiance, and Digital Numbers.....	26
7.4 Comparing Datasets Fairly.....	26
Chapter 8 — Advanced Usage.....	28
8.1 Working with Large Datasets	28
8.2 Accessing and Filtering CubeRecord Objects	28
8.3 Exporting Records to a DataFrame	28
8.4 Manual Band Overrides.....	29
8.5 Integrating with a Machine-Learning Pipeline	29
8.6 Comparing Two Datasets	29
Chapter 9 — HTML Report & CLI	30
9.1 CLI Workflow	30
9.2 Command Syntax	30
9.3 Arguments.....	30
9.4 Important Option Behaviour	31
9.5 Practical CLI Examples.....	31
Single cube — summary only	31
Single cube — plot and save	31
Single cube — supply wavelength values	31
Directory dataset — infer labels and create dashboard	31
Directory dataset — create HTML report	31
Directory dataset — report and dataset dashboard together	31
9.6 CLI Versus Python API.....	31
Chapter 10 — API Reference	33
HyperspectralEDA.__init__().....	33
HyperspectralEDA.load()	33
HyperspectralEDA.load_arrays().....	33
HyperspectralEDA.summary()	33
HyperspectralEDA.get_record()	33
HyperspectralEDA.spectral_signature().....	33
HyperspectralEDA.compute_index().....	33
HyperspectralEDA.pca_scores()	33
HyperspectralEDA.plot()	34
HyperspectralEDA.plot_dataset().....	34
HyperspectralEDA.plot_spectra()	34
HyperspectralEDA.plot_false_colour().....	34
HyperspectralEDA.plot_ndvi()	34
HyperspectralEDA.plot_pca_components().....	34
HyperspectralEDA.plot_band_stats()	34

HyperspectralEDA.plot_spectral_diversity()	34
HyperspectralEDA.report()	34
Chapter 11 — Troubleshooting	36
No valid cubes found	36
Ground-truth masks are analysed as cubes	36
Wavelength length does not match band count	36
EVI is extremely large	36
GLCM is blank or unavailable	36
PCA is blank	36
Spectral diversity says at least two cubes are required	36
The .mat loader chooses the wrong array	36
ENVI or GeoTIFF loading fails	36
The Word TOC is blank	36
The Word output is permission denied	36
Chapter 12 — Roadmap	37
Version 1.0.0 — Current Documented Release	37
Roadmap	37
Appendix A — Complete CubeRecord Field Reference	38
Appendix B — summary() Schema	39
Appendix C — Glossary	40
Appendix D — Acknowledgements	41

Chapter 1 — Introduction

1.1 What is HyperspectralEDA?

HyperspectralEDA is the hyperspectral and multispectral analysis module of VisEDA. It accepts a single data cube, a directory of cubes, an explicit file list, or in-memory NumPy arrays. Every cube is normalised to the convention (H, W, B): image height, image width, and spectral bands. At the individual-cube level, the module computes metadata, spatial maps, per-band statistics, spectral quality indicators, spectral indices, texture measures, and PCA variance. At the dataset level, it aggregates compatible cubes to reveal distributions, class balance, cross-cube spectra, and spectral diversity.

1.2 Why Hyperspectral EDA Matters

- **Band integrity:** A cube may contain bands with near-zero variance, saturation, severe noise, or sensor artefacts.
- **Radiometric consistency:** Cubes from different sensors or processing stages may use reflectance, radiance, or raw digital numbers on incompatible scales.
- **Wavelength alignment:** Spectral indices are only meaningful when band positions correspond to the intended wavelengths.
- **Dimensionality:** Hundreds of bands are often highly redundant; PCA shows how many components explain most variance.
- **Spectral similarity:** Near-identical cubes can produce data leakage or a misleadingly large training set.
- **Spatial inconsistency:** Different cube dimensions and resolutions require explicit preprocessing decisions.
- **Class imbalance:** Folder-based labels may be unevenly represented, biasing downstream classifiers.
- **Corrupt or auxiliary files:** Ground-truth masks and metadata arrays can be mistaken for cubes unless the dataset is organised correctly.

1.3 The Hyperspectral Cube

A hyperspectral cube combines two spatial dimensions with one spectral dimension. For a cube C with shape $H \times W \times B$, $C[i, j, b]$ is the measured value at row i , column j , and band b . The implementation expects or converts every cube to this H-W-B layout. A two-dimensional array is treated as a single-band cube with shape $H \times W \times 1$.

$$C \in \mathbb{R}^{(H \times W \times B)}$$

$H = \text{height}, W = \text{width}, B = \text{spectral bands}$

Note: The loader does not automatically distinguish a ground-truth label mask from a one-band image. Store `*_gt.mat` masks outside the analysis directory, or filter them before loading.

1.4 Design Philosophy

- **One class for single and multiple cubes.** The same HyperspectralEDA object supports a deep dive into one cube and aggregate dataset analysis.
- **Progressive disclosure.** `summary()` provides structured statistics; `plot()` and `plot_dataset()` provide visual interpretation; `report()` creates a shareable HTML file.
- **Lazy optional dependencies.** GLCM, PCA, ENVI, GeoTIFF, and MATLAB support are imported only when required.
- **Memory-aware computation.** Inter-band correlation, PCA, and GLCM use deterministic pixel sampling when cubes are large.
- **Reproducibility.** Random sampling uses a fixed NumPy random generator seed of 0 in the current implementation.
- **Transparent records.** Every cube is represented by a CubeRecord object containing all computed fields.

1.5 Installation

Install the core package and the dependencies needed for the formats and analyses you intend to use:

```
pip install viseda numpy matplotlib
```

```
# MATLAB .mat files
pip install scipy

# PCA and GLCM texture
pip install scikit-learn scikit-image opencv-python

# ENVI .hdr / .bil / .bip / .bsq
pip install spectral

# Multi-band GeoTIFF
pip install rasterio

# Editable Word documentation generator
pip install python-docx pillow
```

Note: When `compute_glcm=False` or `compute_pca=False`, the corresponding optional analysis is skipped. The remainder of HyperspectralEDA continues to work.

1.6 Supported Data Formats

Table 1. Supported HyperspectralEDA input formats.

Extension	Reader	Expected behaviour	Dependency
.mat	scipy.io.loadmat	Selects the largest non-private array in the MATLAB file.	scipy
.npy	numpy.load	Loads one array and converts it to float32.	numpy
.npz	numpy.load	Uses the first key in the archive.	numpy
.hdr/.bil/.bip/.bsq/.envi	spectral.open_image	Reads ENVI-compatible data and normalises to H-W-B.	spectral
.tif/.tiff	rasterio.open	Reads B-H-W and transposes to H-W-B.	rasterio

1.7 Wavelength Conventions

The optional wavelengths array should contain one wavelength value in nanometres for each band. When supplied, spectral plots use a physical wavelength axis and spectral indices automatically select the nearest bands to approximately 490 nm (blue), 560 nm (green), 670 nm (red), and 850 nm (near infrared). When wavelengths are omitted, the module uses band indices $0 \dots B-1$ and the automatic spectral-index band selection becomes an index-space approximation.

Note: A wavelength array whose length differs from the current cube band count should be corrected before interpretation. Different datasets such as PaviaU, Indian Pines, and Salinas have different band counts; one global wavelength array may therefore be inappropriate for a mixed directory.

Chapter 2 — Quick Start

2.1 Analyse a Single Cube

```
import numpy as np
from viseda import HyperspectralEDA

# Example wavelength axis for a 103-band PaviaU cube
wavelengths = np.linspace(400, 1500, 103)

eda = HyperspectralEDA(
    verbose=True,
    wavelengths=wavelengths,
    compute_glcml=True,
    compute_pca=True,
)
eda.load("PaviaU.mat")

stats = eda.summary()
print(stats["inventory"])
print(stats["spectral_quality"])

eda.plot(cube_index=0, save_path="outputs_hyper/plot.png")
eda.plot_band_stats(cube_index=0,
                    save_path="outputs_hyper/plot_band_stats.png")
```

2.2 Analyse a Directory of Cubes

```
from viseda import HyperspectralEDA

# Recommended dataset structure
# hyperspectral_dataset/
#   city/
#   PaviaU.mat
#   farmland/
#   Indian_pines_corrected.mat
#   forest/
#   Salinas_corrected.mat

eda = HyperspectralEDA(verbose=True, compute_glcml=True, compute_pca=True)
eda.load(
    "hyperspectral_dataset/",
    label_from_parent=True,
    recursive=True,
)

eda.plot_dataset(save_path="outputs_hyper/plot_dataset.png")
eda.plot_false_colour(save_path="outputs_hyper/plot_false_colour.png")
eda.plot_ndvi(save_path="outputs_hyper/plot_ndvi.png")
eda.plot_spectra(save_path="outputs_hyper/plot_spectra.png")
eda.report("outputs_hyper/viseda_hyper_report.html")
```

2.3 Load NumPy Arrays Directly

```
import numpy as np
from viseda import HyperspectralEDA

cube_a = np.load("cube_a.npy") # shape: (H, W, B)
cube_b = np.load("cube_b.npy")

eda = HyperspectralEDA(
    wavelengths=np.linspace(400, 2500, cube_a.shape[2]),
    compute_glcml=False,
    compute_pca=True,
)
eda.load_arrays(
```

```
[cube_a, cube_b],
labels=["farmland", "urban"],
)
eda.plot_dataset()
```

2.4 Get the Summary Dictionary

```
summary = eda.summary()

print(summary["inventory"])
print(summary["spatial"])
print(summary["spectral_stats"])
print(summary["spectral_quality"])
print(summary["spectral_indices"])
print(summary["texture"])
print(summary["pca"])
print(summary["labels"])
```

2.5 Generate All Visualisations

```
eda.plot(cube_index=0)           # single-cube dashboard
eda.plot_dataset()              # full dataset dashboard
eda.plot_spectra(max_cubes=20)  # spectral overlay
eda.plot_false_colour(n=12, cols=4) # false-colour grid
eda.plot_ndvi(n=12, cols=4)      # NDVI maps
eda.plot_pca_components(cube_index=0,
                        n_components=6) # PCA images
eda.plot_band_stats(cube_index=0)  # six per-band curves
eda.plot_spectral_diversity()      # cosine similarity heatmap
```

2.6 Extract a Pixel Spectral Signature

```
wavelengths, signature = eda.spectral_signature(
    cube_index=0,
    row=100,
    col=120,
)

print(wavelengths.shape)
print(signature.shape)
```

2.7 Compute Spectral Index Maps

```
ndvi = eda.compute_index(0, "ndvi")
ndwi = eda.compute_index(0, "ndwi")
evi = eda.compute_index(0, "evi")
savi = eda.compute_index(0, "savi")

print(ndvi.shape) # (height, width)
```

2.8 Obtain PCA Scores

```
scores, variance_ratio = eda.pca_scores(
    cube_index=0,
    n_components=10,
)

print(scores.shape)          # (H*W, 10)
print(variance_ratio)        # explained variance per component
print(variance_ratio.cumsum())
```

2.9 Generate the HTML Report

```
report_path = eda.report("outputs_hyper/viseda_hyper_report.html")
print(report_path)
```

2.10 Command-Line Interface

```
# Basic summary
viseda hyper path/to/cubes/

# Infer labels from parent folders and save an HTML report
viseda hyper path/to/cubes/ --label-from-parent --report outputs_hyper/report.html

# Provide wavelengths saved as a NumPy array
viseda hyper path/to/PaviaU.mat --wavelengths wavelengths.npy --plot --save-plot outputs_hyper/plot.png

# Dataset dashboard rather than the single-cube plot
viseda hyper path/to/cubes/ --label-from-parent --plot --dataset-plot --save-plot outputs_hyper/plot_dataset.png
```

Chapter 3 — The HyperspectralEDA Class

3.1 Constructor

```
eda = HyperspectralEDA(
    verbose=True,
    wavelengths=None,
    max_cubes=None,
    compute_glcmm=True,
    compute_pca=True,
    dropout_threshold=0.01,
    ndvi_nir_band=None,
    ndvi_red_band=None,
    ndwi_green_band=None,
    ndwi_nir_band=None,
)
```

Table 2. HyperspectralEDA constructor parameters.

Parameter	Type	Default	Meaning
verbose	bool	True	Print loading and progress messages.
wavelengths	1-D array or None	None	Band-centre wavelengths in nm. Enables physical-axis plots and automatic index bands.
max_cubes	int or None	None	Maximum cubes analysed during load().
compute_glcmm	bool	True	Compute GLCM texture from the first principal component.
compute_pca	bool	True	Compute up to 20 PCA explained-variance components.
dropout_threshold	float	0.01	A band is flagged when its standard deviation is below this fraction of the standard deviation of band standard deviations.
ndvi_nir_band	int or None	None	Manual NIR band override for NDVI/EVI/SAVI.
ndvi_red_band	int or None	None	Manual red band override for NDVI/EVI/SAVI.
ndwi_green_band	int or None	None	Manual green band override for NDWI.
ndwi_nir_band	int or None	None	Manual NIR band override for NDWI.

3.2 load() — Load Cubes from Files

```
def load( self, source: Union[str, Path, List], labels: Optional[Dict[str, str]] = None, label_from_parent: bool = False, recursive: bool = True, ) -> "HyperspectralEDA":
```

Discovers supported cube files, applies max_cubes, determines labels, computes one CubeRecord per file, and marks unreadable inputs as corrupt rather than terminating the complete dataset analysis.

3.3 load_arrays() — Load In-Memory Cubes

```
def load_arrays( self, arrays: List[np.ndarray], labels: Optional[List[str]] = None, ) -> "HyperspectralEDA":
```

Accepts a list of H-W-B NumPy arrays. Each array is converted to float32 and analysed using the same statistics as file-backed cubes. Array-backed records use names such as <array_0>.

3.4 summary() — Aggregate Statistics

```
def summary(self) -> Dict[str, Any]:
```

Returns inventory, spatial, spectral_stats, spectral_quality, spectral_indices, texture, PCA, and label sections. Compatible band counts are grouped using the dominant band count for cross-cube spectral aggregation.

3.5 get_record() — Access a CubeRecord

```
def get_record(self, index: int = 0) -> CubeRecord:
```

Returns the record at the requested position. This is the primary route for filtering, sorting, exporting, or inspecting per-cube fields.

3.6 spectral_signature() — Read One Pixel Spectrum

```
def spectral_signature( self, cube_index: int = 0, row: int = 0, col: int = 0 ) -> Tuple[np.ndarray, np.ndarray]:
```

Re-reads the selected file-backed cube and returns the wavelength/index axis and spectral values for a single pixel. Coordinates are checked against cube dimensions.

3.7 compute_index() — Create an Index Map

```
def compute_index( self, cube_index: int = 0, index_name: str = "ndvi", ) -> np.ndarray:
```

Computes NDVI, NDWI, EVI, or SAVI for one cube. The result is a two-dimensional map with the cube height and width.

3.8 pca_scores() — Obtain Pixel-Level Components

```
def pca_scores( self, cube_index: int = 0, n_components: int = 10, ) -> Tuple[np.ndarray, np.ndarray]:
```

Reshapes the cube to H×W pixel spectra, runs randomized PCA, and returns component scores and explained-variance ratios.

3.9 report() — Generate HTML

```
def report(self, output_path: str = "vised_hyperspectral_report.html") -> str:
```

Writes a self-contained HTML report based on summary(). The report contains inventory, spatial, and spectral statistics; spectral quality; indices; PCA; texture; labels; and dependency-free styling.

3.10 Object State and Method Chaining

load() and load_arrays() return self, allowing method chaining. summary(), plotting methods, report(), spectral_signature(), compute_index(), and pca_scores() require a loaded object. Calling them before loading raises a RuntimeError through the internal _check_loaded() guard.

```
eda = (
    HyperspectralEDA(verbose=True)
    .load("dataset/", label_from_parent=True)
)

summary = eda.summary()
```

Chapter 4 — The CubeRecord Object

Each analysed cube is represented by a lightweight CubeRecord. The object uses `__slots__`, so its supported fields are explicit and memory overhead is reduced. Fields are initialised to `None` and populated when the corresponding analysis succeeds.

4.1 Identity and Shape Fields

Field	Type	Meaning
path	str	File path or <array_i>.
label	str or None	Class label from a mapping or parent directory.
file_ext	str	Extension such as .mat, .npy, .tif, or array.
file_size_kb	float or None	File size for file-backed cubes.
height	int	Spatial height H.
width	int	Spatial width W.
bands	int	Spectral band count B.
dtype	str	Internal cube dtype after conversion, normally float32.
is_corrupt	bool	True when reading or analysis failed.

4.2 Global and Per-Band Fields

Field	Type	Meaning
global_mean/global_std	float	Overall cube mean and standard deviation.
global_min/global_max	float	Overall minimum and maximum.
dynamic_range	float	global_max – global_min.
band_means	array (B,)	Mean of every spectral band.
band_stds	array (B,)	Standard deviation of every band.
band_mins/band_maxs	array (B,)	Per-band extrema.
band_snr	array (B,)	Per-band mean divided by standard deviation.
band_noise_mad	array (B,)	Per-band median absolute deviation.
band_saturation_frac	array (B,)	Fraction of pixels equal to the cube global maximum.

4.3 Spectral Quality and Spatial Fields

Field	Type	Meaning
spectral_smoothness	float	Mean absolute difference between adjacent band means.
inter_band_corr	float	Mean off-diagonal correlation between bands.
n_dropout_bands	int	Bands below the configured variability threshold.
snr_mean	float	Mean of band_snr.
spatial_mean_map	array (H,W)	Mean across bands for each pixel.
spatial_std_map	array (H,W)	Standard deviation across bands for each pixel.

4.4 Spectral Indices, PCA, and Texture

Field	Type	Meaning
ndvi_mean/ndvi_std	float	Summary of the NDVI map.
ndwi_mean/ndwi_std	float	Summary of the NDWI map.
evi_mean/evi_std	float	Summary of the EVI map.
savi_mean/savi_std	float	Summary of the SAVI map.
pca_variance_ratio	array	Explained variance for up to 20 components.
glcm_contrast	float	Texture contrast on PC1.
glcm_dissimilarity	float	Texture dissimilarity on PC1.

Field	Type	Meaning
glcm_homogeneity	float	Texture homogeneity on PC1.
glcm_energy	float	GLCM energy on PC1.
glcm_correlation	float	GLCM correlation on PC1.
glcm_asm	float	Angular second moment on PC1.

Chapter 5 — What Is Analysed

5.1 Dataset Inventory

The inventory section records total, valid, and corrupt cube counts; corrupt paths; distributions of band counts and file formats; and labels. A mixed band-count distribution is not automatically an error, but only cubes sharing the dominant band count contribute to cross-cube spectra and spectral-diversity analysis.

5.2 Spatial Dimensions and File Size

HyperspectralEDA summarises cube heights, widths, band counts, and file sizes using count, mean, standard deviation, minimum, quartiles, median, and maximum. These statistics identify mixed spatial resolutions, inconsistent scene extents, and unexpectedly small files.

5.3 Global Radiometric Statistics

`global_mean` and `global_std` describe the overall radiometric scale of each cube. `global_min` and `global_max` identify the observed range, while `dynamic_range` is the difference between the extrema. These values should not be compared blindly across reflectance, radiance, and raw digital-number datasets.

$$\text{Dynamic range} = \max(C) - \min(C)$$

5.4 Per-Band Mean, Standard Deviation, Minimum, and Maximum

For every band b , HyperspectralEDA computes mean, standard deviation, minimum, and maximum over the $H \times W$ spatial plane. The band-mean curve is the cube's average spectral signature. Band-standard-deviation peaks indicate wavelengths with strong spatial variability, while minima may reveal low-information or defective bands.

5.5 Signal-to-Noise Ratio

$$SNR_b = \mu_b / (\sigma_b + \epsilon)$$

μ_b and σ_b are the spatial mean and standard deviation of band b ; $\epsilon = 10^{-9}$.

The implementation uses a simple mean-to-standard-deviation ratio. `snr_mean` is the average across bands. This is a descriptive signal-consistency measure rather than a calibrated instrument SNR. Low values can indicate weak signal, high spatial variation, noise, or a mean near zero.

5.6 Robust Noise Using Median Absolute Deviation (*MAD*)

$$MAD_b = \text{median}(|C_b - \text{median}(C_b)|)$$

Median absolute deviation is more robust to isolated extreme pixels than standard deviation. The `plot_band_stats()` dashboard exposes the full per-band **MAD** curve, helping identify noisy wavelength regions.

5.7 Saturation Fraction

$$\text{Saturation fraction}_b = \text{count}(C_b = \text{global max}) / (H \times W)$$

The current implementation defines saturation relative to the cube's global maximum. A high value means many pixels in a band are pinned at that maximum. Because the maximum is cube-dependent, interpretation should consider sensor bit depth and preprocessing.

5.8 Spectral Smoothness

$$\text{Smoothness} = \text{mean}(|\mu_{(b+1)} - \mu_b|)$$

This scalar measures the average step between adjacent band means. A low value indicates a smooth mean spectrum; a high value indicates rapid changes, strong absorption features, calibration discontinuities, or mismatched band ordering. Absolute values depend on radiometric scale.

5.9 Inter-Band Correlation

Pixel spectra are reshaped to an $N \times B$ matrix. When N exceeds 5,000 pixels, a deterministic 5,000-pixel sample is used. The $B \times B$ correlation matrix is calculated, and the mean of all off-diagonal entries becomes `inter_band_corr`. High correlation suggests spectral redundancy; low or negative values indicate more independent band behaviour or potential artefacts.

5.10 Dropout Band Detection

$$\text{Dropout if } \sigma_b < \text{dropout_threshold} \times \text{std}(\{\sigma_1, \dots, \sigma_b\})$$

The default threshold is 0.01. A dropout band has exceptionally low spatial variability relative to the spread of band standard deviations. The count is stored as `n_dropout_bands`. Constant or almost constant bands are strong candidates for removal.

Note: This implementation uses the standard deviation of the per-band standard deviations, not the overall cube standard deviation. Interpret the threshold accordingly when customising `dropout_threshold`.

5.11 Spatial Mean and Spatial Standard-Deviation Maps

`spatial_mean_map` averages all bands at every pixel. It gives a panchromatic-like structural view. `spatial_std_map` measures how strongly each pixel varies over wavelengths, highlighting spectrally distinctive materials or boundaries.

5.12 Spectral Indices

When appropriate bands can be selected, the module computes four common indices. Band positions are chosen by the nearest wavelength or by explicit constructor overrides.

$$\text{NDVI} = (\text{NIR} - \text{Red}) / (\text{NIR} + \text{Red} + \epsilon)$$

$$\text{NDWI} = (\text{Green} - \text{NIR}) / (\text{Green} + \text{NIR} + \epsilon)$$

$$\text{EVI} = 2.5(\text{NIR} - \text{Red}) / (\text{NIR} + 6\text{Red} - 7.5\text{Blue} + 1 + \epsilon)$$

$$\text{SAVI} = 1.5(\text{NIR} - \text{Red}) / (\text{NIR} + \text{Red} + 0.5 + \epsilon)$$

Note: NDVI and related thresholds assume reflectance-like inputs. Raw digital numbers or radiance can produce unusual values, especially EVI when the denominator approaches zero. The test suite therefore checks finiteness rather than forcing synthetic reflectance ranges for real `.mat` datasets.

5.13 Principal Component Analysis

PCA reshapes the cube to $H \times W$ pixel spectra and fits up to $\min(20, B, \text{number of pixels})$ components. If a cube exceeds 10,000 pixels, a fixed 10,000-pixel sample is used for fitting. `pca_variance_ratio` stores the explained variance. The summary reports the mean component profile across cubes and the mean number of components needed to reach 95% cumulative variance.

5.14 GLCM Texture on the First Principal Component

Texture is computed on a first-principal-component image because PC1 condenses the dominant spatial-spectral information into one greyscale plane. The PC1 image is resized to 128×128 , quantised to 64 grey levels, and analysed at distance 1 and angles 0° , 45° , 90° , and 135° . The resulting properties are averaged over angles.

GLCM feature	Interpretation
Contrast	Emphasises strong local intensity differences.
Dissimilarity	Measures average absolute grey-level difference.
Homogeneity	High when co-occurrences concentrate near the diagonal.
Energy	Square root of angular second moment; high for regular texture.
Correlation	Measures linear dependency between neighbouring grey levels.
ASM	Sum of squared GLCM probabilities.

5.15 Cross-Cube Mean Spectrum

The dominant band count is the most frequent B value in the dataset. Cubes with that band count are stacked by band_means to calculate a dataset mean spectrum and standard-deviation envelope. This avoids invalid stacking of incompatible spectral dimensions.

5.16 Spectral Diversity

$$\text{cosine similarity}(i,j) = (\mathbf{s}_i \cdot \mathbf{s}_j) / (\|\mathbf{s}_i\| \|\mathbf{s}_j\|)$$

Mean spectra from up to 50 cubes sharing the dominant band count are normalised and compared by cosine similarity. Values close to 1 indicate very similar spectral shapes. The heatmap is especially useful for detecting redundant scenes, repeated samples, or class clusters.

5.17 Corrupt-File Detection

Read or analysis failures are caught per cube. The corresponding CubeRecord is marked is_corrupt=True and remains in the inventory, allowing the dataset analysis to complete. The summary records corrupt_paths for follow-up.

Chapter 6 — Visualisations

6.1 plot() — Single-Cube Deep-Dive Dashboard

```
eda.plot(
    cube_index=0,
    figsize=(22, 20),
    save_path="outputs_hyper/plot.png",
    dpi=150,
    rgb_bands=None,
)
```

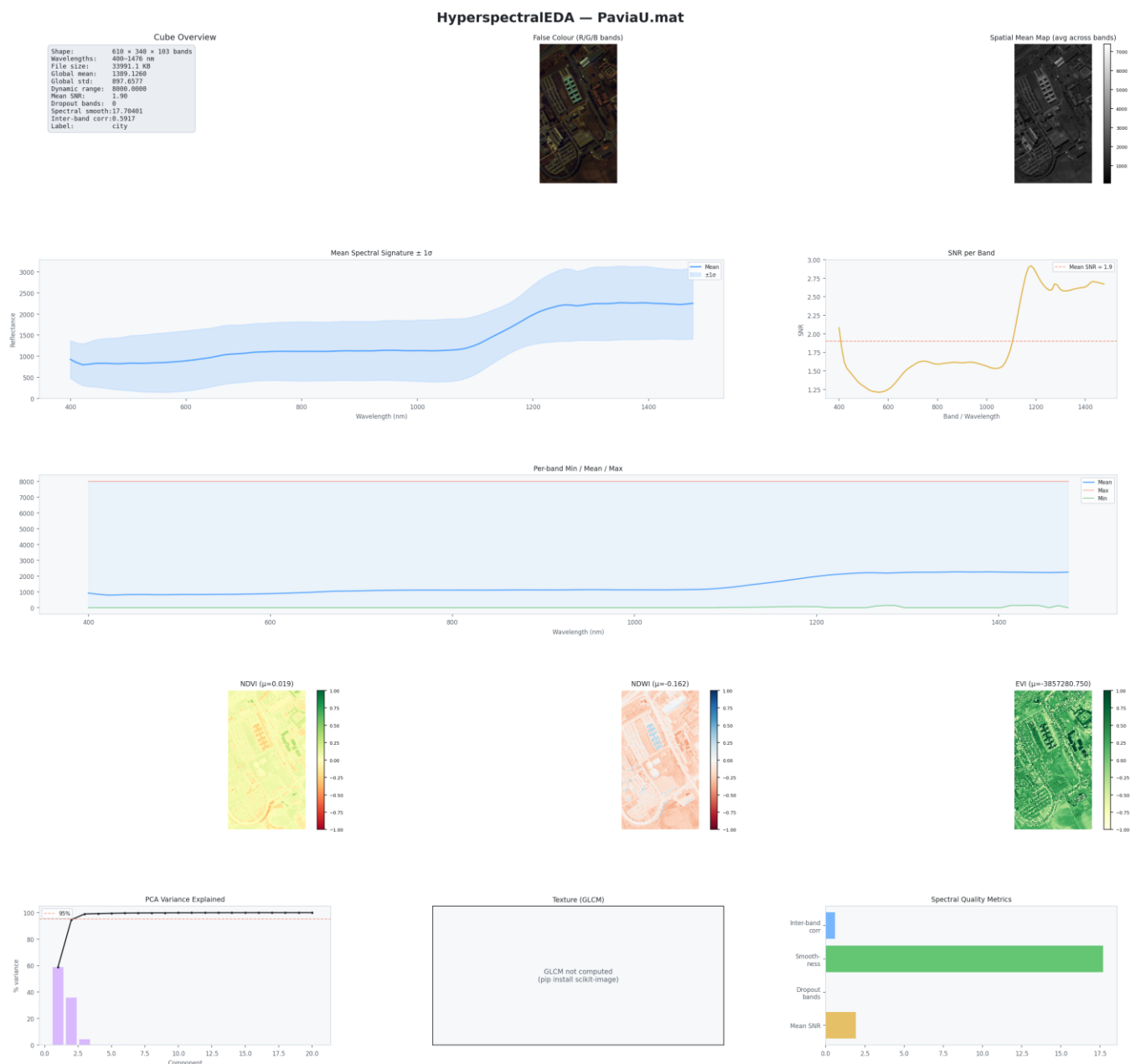


Figure 6-1. HyperspectralEDA single-cube dashboard

The dashboard combines cube metadata, false colour, a spatial mean map, spectral curves, indices, PCA, texture, and quality indicators for one cube.

Cube Overview

Shows shape, wavelength range, file size, global mean and standard deviation, dynamic range, mean SNR, dropout bands, spectral smoothness, inter-band correlation, and label.

False Colour

Uses three selected bands as red, green, and blue channels after percentile-style normalisation. `rgb_bands` can override automatic selection.

Spatial Mean Map

Displays the mean across all bands for every pixel, providing a panchromatic structural overview.

Mean Spectral Signature $\pm 1\sigma$

Plots `band_means` and an envelope based on band standard deviations.

SNR per Band

Shows `band_snr` and a horizontal line at the cube mean SNR.

Per-Band Min / Mean / Max

Compares extrema and mean across the full wavelength axis.

NDVI, NDWI, and EVI maps

Displays spatial index patterns. The figure title includes each map's mean value.

PCA Variance

Shows individual component variance and cumulative explained variance with a 95% reference line.

Texture (GLCM)

A radar plot of available GLCM properties; a message appears when GLCM is unavailable.

Spectral Quality Metrics

Summarises inter-band correlation, smoothness, dropout bands, and mean SNR in one bar chart.

6.2 `plot_dataset()` — Dataset Analysis Dashboard

```
eda.plot_dataset(  
    figsize=(24, 24),  
    save_path="outputs_hyper/plot_dataset.png",  
    dpi=150,  
)
```

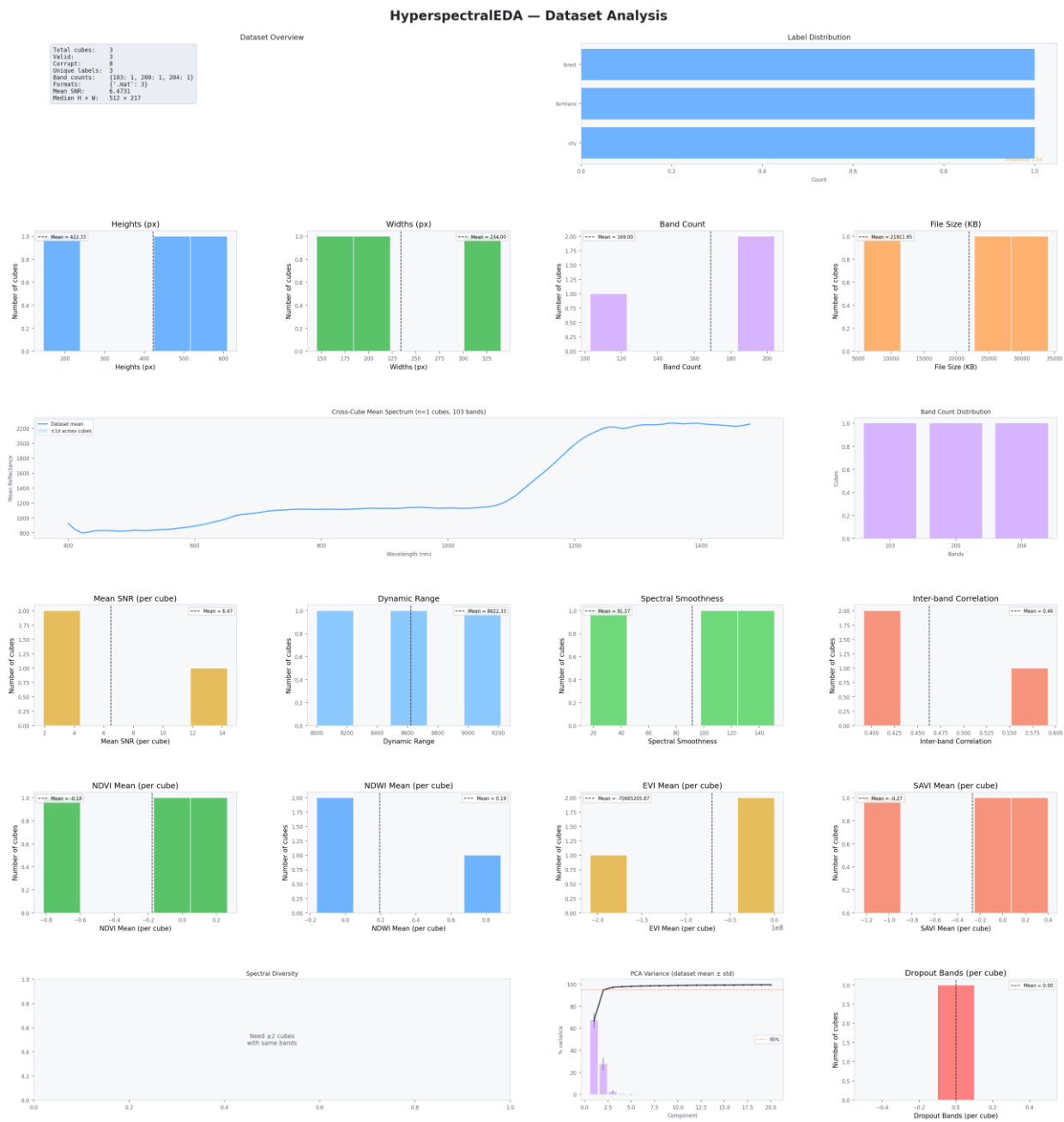


Figure 6-2. HyperspectralEDA dataset dashboard

The six-row dashboard summarises inventory, labels, spatial dimensions, band counts, cross-cube spectra, spectral quality, indices, PCA, dropout bands, and spectral diversity.

Dashboard row	Content
Row 0	Dataset overview card and label distribution.
Row 1	Height, width, band-count, and file-size distributions.
Row 2	Cross-cube mean spectrum with standard-deviation envelope and band-count distribution.
Row 3	Mean SNR, dynamic range, spectral smoothness, and inter-band correlation distributions.
Row 4	NDVI, NDWI, EVI, and SAVI per-cube mean distributions.
Row 5	Spectral diversity, dataset PCA variance, and dropout-band distribution.

6.3 plot_spectra() — Spectral Overlay

```
eda.plot_spectra(
    max_cubes=20,
    figsize=(14, 6),
    save_path="outputs_hyper/plot_spectra.png",
    dpi=150,
)
```

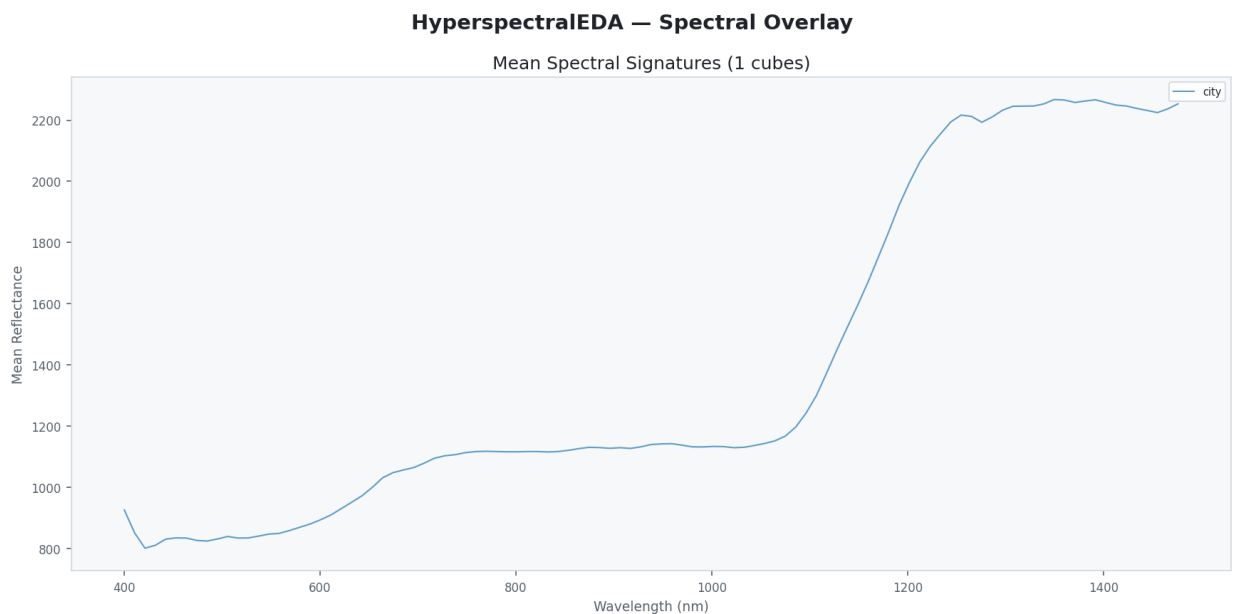


Figure 6-3. Mean spectral signatures

Each curve is the per-band mean of one compatible cube. Labels are used in the legend when available. The plot is limited to cubes sharing the dominant band count.

Parallel curves with similar shape but different magnitude indicate comparable spectral behaviour on different radiometric scales. Divergent shapes can reveal different land-cover classes, sensors, preprocessing, or defective cubes.

6.4 plot_false_colour() — False-Colour Previews

```
eda.plot_false_colour(
    n=12,
    cols=4,
    rgb_bands=None,
    save_path="outputs_hyper/plot_false_colour.png",
)
```

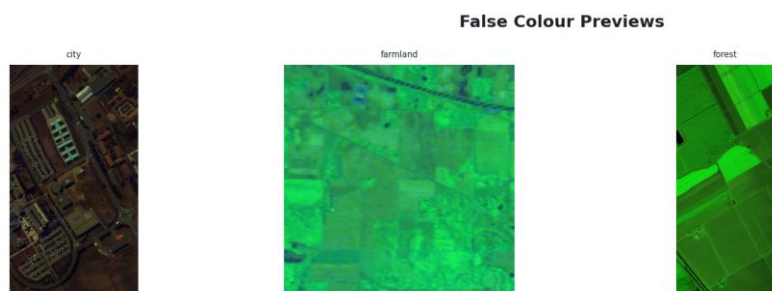


Figure 6-4. False-colour preview grid

Representative false-colour composites for multiple cubes. Parent-folder labels or file names identify each panel.

False-colour images are qualitative. The displayed colours depend on band selection and normalisation; they should not be interpreted as calibrated true-colour imagery unless the chosen bands correspond to visible red, green, and blue wavelengths.

6.5 `plot_ndvi()` — NDVI Map Grid

```
eda.plot_ndvi(
    n=12,
    cols=4,
    save_path="outputs_hyper/plot_ndvi.png",
)
```

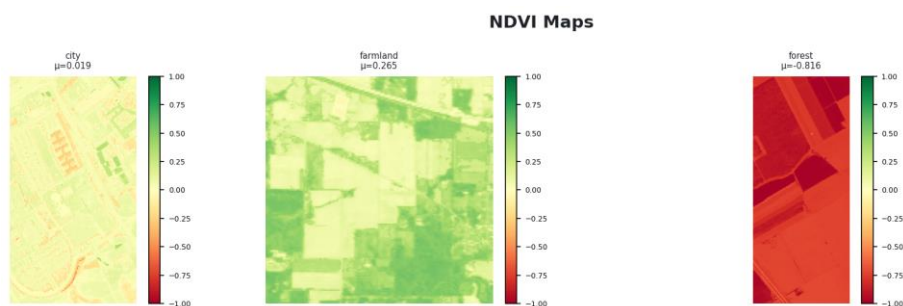


Figure 6-5. NDVI maps

NDVI maps for several cubes, each displayed on a fixed -1 to 1 colour scale with the cube-level mean shown in the title.

Note: Negative or extreme NDVI values can be valid for non-vegetated surfaces, but incorrect wavelength mapping or raw digital numbers can also distort the index.

6.6 `plot_pca_components()` — PCA Component Images

```
eda.plot_pca_components(
    cube_index=0,
    n_components=6,
    save_path="outputs_hyper/plot_pca_components.png",
)
```

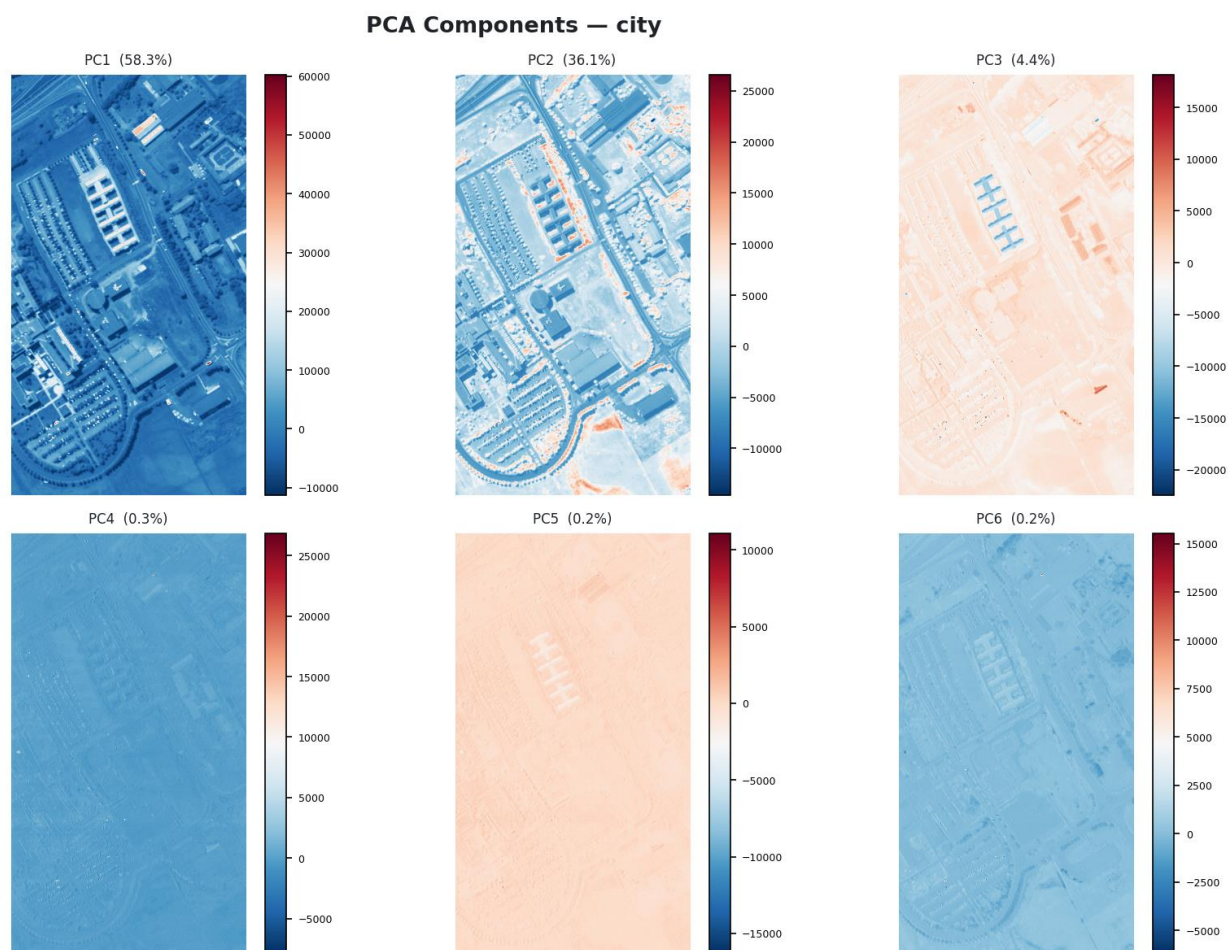


Figure 6-6. Principal component images

The first six PCA score images. Each title reports the percentage of spectral variance explained by the component.

PC1 normally captures overall brightness and major material differences. Later components contain progressively subtler spectral contrasts and may reveal boundaries, rare materials, noise, or striping. Component signs are arbitrary; blue versus red does not imply a universal physical meaning.

6.7 `plot_band_stats()` — Per-Band Diagnostics

```
eda.plot_band_stats(
    cube_index=0,
    figsize=(16, 10),
    save_path="outputs_hyper/plot_band_stats.png",
)
```

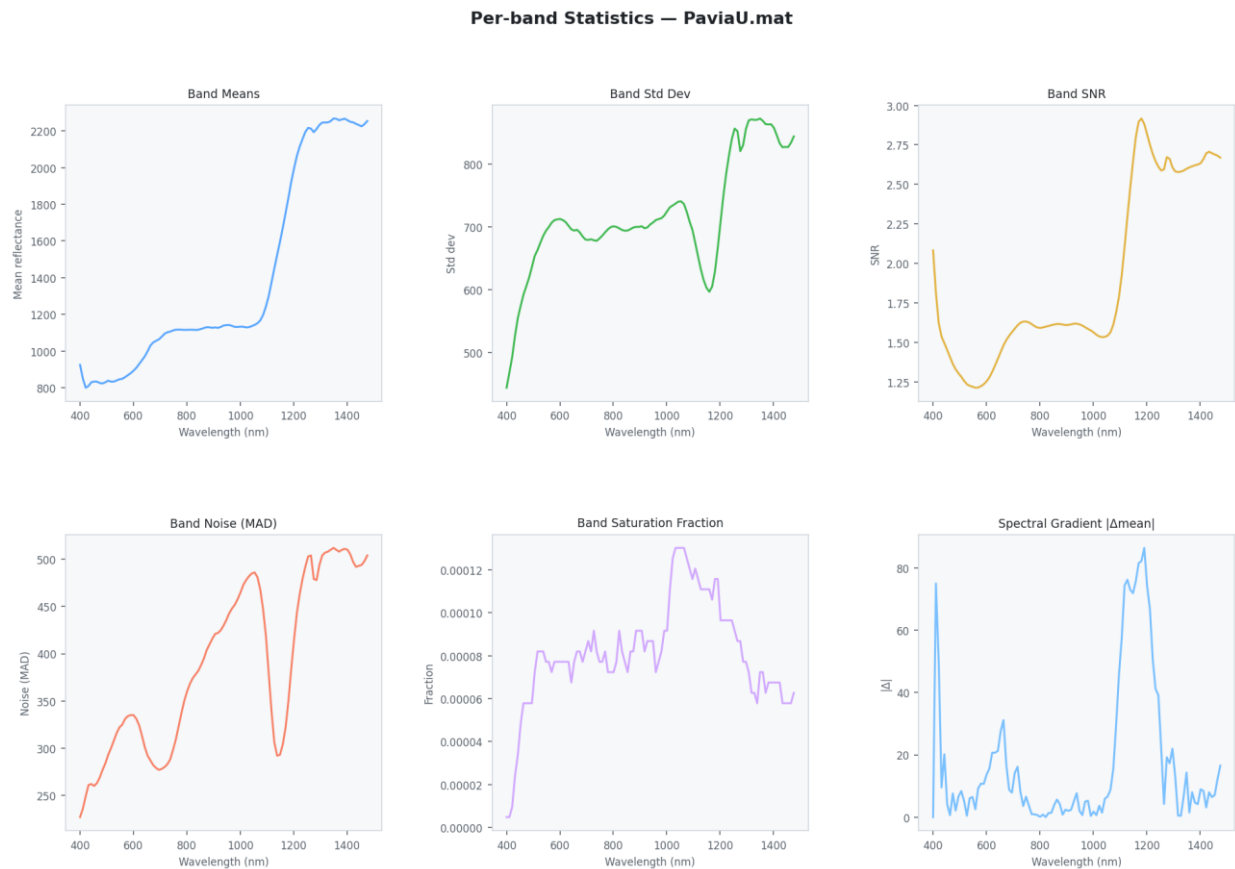


Figure 6-7. Per-band statistics

Six line charts show band mean, band standard deviation, band SNR, robust noise MAD, saturation fraction, and the absolute gradient of adjacent band means.

Panel	Diagnostic value
Band Means	Average spectral signature and broad absorption or response patterns.
Band Std Dev	Spatial variability by wavelength.
Band SNR	Bands with weak mean relative to spatial variation.
Band Noise (MAD)	Robust estimate of within-band spread/noise.
Band Saturation Fraction	Bands with many pixels pinned at the cube maximum.
Spectral Gradient Δmean	Abrupt changes between neighbouring band means.

6.8 plot_spectral_diversity() — Cosine Similarity

```
eda.plot_spectral_diversity(  
    figsize=(10, 8),  
    save_path="outputs_hyper/plot_spectral_diversity.png",  
)
```

Figure omitted: Figure file for 'plot_spectral_diversity' was not found. Generate the plot first or pass --image-dir. Expected one of: plot_spectral_diversity.png, plot_spectral_diversity(1).png.

Chapter 7 — Interpreting Results and Common Findings

7.1 Red Flags to Review

Table 3. Common HyperspectralEDA findings and responses.

Finding	Possible meaning	Recommended action
Corrupt cubes	Unreadable file, unsupported structure, missing dependency, or auxiliary data rather than a cube.	Inspect <code>corrupt_paths</code> ; verify dependencies and folder contents.
Mixed band counts	Different sensors, preprocessed variants, or accidental masks.	Analyse compatible groups separately or resample to a common wavelength grid.
<code>n_dropout_bands > 0</code>	Constant or near-constant bands.	Inspect band indices and consider removing them.
Low mean SNR	Weak signal, strong spatial variability, noise, or scale near zero.	Review band curves; compare only within similar radiometric scales.
High spectral smoothness	Abrupt spectral changes, band-order problems, or real absorption features.	Inspect spectral gradient and wavelength metadata.
Very high inter-band correlation	Substantial spectral redundancy.	Use PCA or band-selection methods.
Very low/negative inter-band correlation	Independent spectral structure, noise, or calibration problems.	Inspect correlation and per-band statistics.
Extreme EVI	Denominator close to zero or non-reflectance data.	Check wavelength mapping and radiometric preprocessing.
PCA reaches 95% in 1–2 components	Strong redundancy or brightness-dominated variation.	Inspect later components before aggressive reduction.
Large low-similarity blocks	Distinct classes/sensors or inconsistent preprocessing.	Verify labels and acquisition conditions.

7.2 Interpreting the Attached Example Outputs

The supplied example dataset contains city, farmland, and forest cubes with different spatial dimensions and band counts. The dataset dashboard correctly reports three valid cubes and separate band-count categories. Because only one cube has the dominant 103-band configuration, the cross-cube mean spectrum and spectral-diversity panels cannot represent all three cubes simultaneously. This illustrates why wavelength-compatible grouping is essential.

The PaviaU single-cube output shows a 610×340×103 cube. Its mean spectrum changes strongly across the wavelength axis, while the PCA panel indicates that the first two components explain most variance. The GLCM panel reports that texture was not computed in that run, which can occur when `compute_glcm=False` or an optional dependency is unavailable.

The example EVI mean is numerically extreme. This is not automatically a software error: EVI contains a denominator that can approach zero, and public MATLAB datasets are frequently stored as digital numbers rather than atmospherically corrected reflectance. Index maps should therefore be interpreted only after confirming the radiometric scale and wavelength assignments.

7.3 Reflectance, Radiance, and Digital Numbers

Thresholds commonly quoted for vegetation indices assume surface reflectance. Radiance retains illumination and atmospheric effects, while raw digital numbers are sensor-scaled integers. HyperspectralEDA does not automatically convert among these physical quantities. The user must supply appropriately calibrated data for physically meaningful index interpretation.

7.4 Comparing Datasets Fairly

- Compare cubes only after confirming consistent units and preprocessing.
- Use the same wavelength grid or resample spectra before distance calculations.

- Separate sensors with different spectral response functions.
- Compare SNR and smoothness within a sensor/acquisition group rather than across arbitrary scales.
- Use class-aware spectral overlays to check whether between-class differences exceed within-class variation.

Chapter 8 — Advanced Usage

8.1 Working with Large Datasets

```
# Quick overview of the first 100 cubes
eda = HyperspectralEDA(
    max_cubes=100,
    compute_glm=False,
    compute_pca=False,
)
eda.load("large_dataset/", label_from_parent=True)

# Re-enable expensive analyses for a representative subset
detailed = HyperspectralEDA(
    max_cubes=12,
    compute_glm=True,
    compute_pca=True,
).load("large_dataset/", label_from_parent=True)
```

GLCM and PCA are the most computationally expensive analyses. The implementation already samples pixels for correlation, GLCM fitting, and PCA fitting, but `max_cubes` and the compute flags provide the largest speed gains.

8.2 Accessing and Filtering CubeRecord Objects

```
records = [r for r in eda._records if not r.is_corrupt]

# Lowest-SNR cubes
low_snr = sorted(records, key=lambda r: r.snr_mean or float("inf"))[:10]
for rec in low_snr:
    print(rec.path, rec.snr_mean)

# Cubes containing dropout bands
dropout = [r for r in records if (r.n_dropout_bands or 0) > 0]

# Spectrally rough cubes
rough = sorted(
    records,
    key=lambda r: r.spectral_smoothness or 0,
    reverse=True,
)[:10]
```

8.3 Exporting Records to a DataFrame

```
import pandas as pd

rows = []
for r in eda._records:
    rows.append({
        "path": r.path,
        "label": r.label,
        "height": r.height,
        "width": r.width,
        "bands": r.bands,
        "global_mean": r.global_mean,
        "global_std": r.global_std,
        "snr_mean": r.snr_mean,
        "spectral_smoothness": r.spectral_smoothness,
        "inter_band_corr": r.inter_band_corr,
        "n_dropout_bands": r.n_dropout_bands,
        "ndvi_mean": r.ndvi_mean,
        "is_corrupt": r.is_corrupt,
    })

df = pd.DataFrame(rows)
df.to_csv("outputs_hyper/hyperspectral_inventory.csv", index=False)
```

8.4 Manual Band Overrides

```
# Use known sensor band indices instead of wavelength auto-selection
eda = HyperspectralEDA(
    ndvi_nir_band=80,
    ndvi_red_band=55,
    ndwi_green_band=30,
    ndwi_nir_band=80,
)
eda.load("cube.npy")
```

Note: The implementation uses expressions such as `self.ndvi_nir` or `auto_band`. Therefore, an override value of 0 is treated as false and falls back to automatic selection. Avoid using band index 0 as an override without updating that implementation to test against None explicitly.

8.5 Integrating with a Machine-Learning Pipeline

```
summary = eda.summary()

# Example quality gate
if summary["inventory"]["corrupt_cubes"] > 0:
    raise RuntimeError("Remove or repair corrupt cubes before training")

if summary["spectral_quality"]["n_dropout_bands"]["max"] > 0:
    print("Review dropout bands before model development")

# PCA-based dimensionality decision
n95 = summary["pca"].get("n_components_95pct_mean")
print("Mean components needed for 95% variance:", n95)
```

8.6 Comparing Two Datasets

```
train = HyperspectralEDA(wavelengths=w1).load("train/", label_from_parent=True)
test = HyperspectralEDA(wavelengths=w1).load("test/", label_from_parent=True)

s_train = train.summary()
s_test = test.summary()

print("Train SNR:", s_train["spectral_quality"]["snr_mean"]["mean"])
print("Test SNR:", s_test["spectral_quality"]["snr_mean"]["mean"])
print("Train smoothness:", s_train["spectral_quality"]["spectral_smoothness"]["mean"])
print("Test smoothness:", s_test["spectral_quality"]["spectral_smoothness"]["mean"])
```

Chapter 9 — HTML Report & CLI

`report()` generates a self-contained HTML document. The attached implementation uses a light VisEDA theme and card-based responsive layout. It can be viewed locally without an internet connection and safely shared as one file.

Table 4. Sections in the HyperspectralEDA HTML report.

Report section	Contents
Inventory	Total, valid, corrupt counts; label, band-count, and format distributions.
Spatial	Height, width, bands, and file-size descriptive statistics.
Spectral Statistics	Global mean, global standard deviation, and dynamic range.
Spectral Quality	Mean SNR, spectral smoothness, inter-band correlation, and dropout bands.
Spectral Indices	NDVI, NDWI, EVI, and SAVI per-cube summaries when available.
PCA	Mean variance profile and number of components required for 95% variance.
Texture	GLCM summaries when computed.

```
eda.report("outputs_hyper/viseda_hyper_report.html")
```

9.1 CLI Workflow

The current VisEDA command-line interface exposes the `hyper` subcommand for analysing either one hyperspectral cube or a directory of cubes. The command loads the supplied data, prints the HyperspectralEDA summary as formatted JSON, optionally writes the self-contained HTML report, and can render either the single-cube dashboard or the dataset-level dashboard.

The CLI is intentionally narrower than the full Python API. It is designed for the most common inspection workflow, while specialised constructor controls, individual plotting methods, and manual band overrides remain available through HyperspectralEDA in Python.

9.2 Command Syntax

```
usage: viseda hyper PATH [--wavelengths FILE]
                        [--label-from-parent]
                        [--report FILE]
                        [--plot]
                        [--dataset-plot]
                        [--save-plot FILE]
```

9.3 Arguments

Table 5. Arguments exposed by the current `viseda hyper` command.

Argument	Meaning
PATH	Path to a single hyperspectral file or a directory containing hyperspectral cubes.
--wavelengths FILE	Path to a NumPy .npy file containing one wavelength value for each spectral band.
--label-from-parent	Use the immediate parent directory name as the label for each cube.
--report FILE	Generate the self-contained HyperspectralEDA HTML report at the specified output path.
--plot	Generate a visualisation. By default this calls <code>plot()</code> and therefore renders the single-cube dashboard.
--dataset-plot	When used together with <code>--plot</code> , call <code>plot_dataset()</code> instead of <code>plot()</code> .
--save-plot FILE	Save the visualisation selected by <code>--plot</code> to the specified file. This option does not independently trigger plotting.

9.4 Important Option Behaviour

Important: `--dataset-plot` only changes which plotting method is used after `--plot` has been supplied. On its own, `--dataset-plot` does not create a figure.

Important: `--save-plot` specifies the output path for the selected plot, but it does not independently generate a plot. Use it together with `--plot`.

In practical terms, the CLI first performs loading and summary generation. If `--report` is present, `report()` is called. If `--plot` is present, the CLI then calls `plot_dataset()` when `--dataset-plot` is also present; otherwise it calls `plot()`. The value supplied through `--save-plot` is passed to that plotting method as `save_path`.

```
if args.report:
    eda.report(args.report)

if args.plot:
    if args.dataset_plot:
        eda.plot_dataset(save_path=args.save_plot)
    else:
        eda.plot(save_path=args.save_plot)
```

9.5 Practical CLI Examples

Single cube — summary only

```
visedata hyper "C:/data/PaviaU.mat"
```

Single cube — plot and save

```
visedata hyper "C:/data/PaviaU.mat" --plot --save-plot "outputs_hyper/plot.png"
```

Single cube — supply wavelength values

```
visedata hyper "C:/data/PaviaU.mat" --wavelengths "C:/data/wavelengths.npy" --plot --save-plot
"outputs_hyper/plot.png"
```

Directory dataset — infer labels and create dashboard

```
visedata hyper "C:/data/hyperspectral_dataset" --label-from-parent --plot --dataset-plot --save-plot
"outputs_hyper/plot_dataset.png"
```

Directory dataset — create HTML report

```
visedata hyper "C:/data/hyperspectral_dataset" --label-from-parent --report
"outputs_hyper/visedata_hyper_report.html"
```

Directory dataset — report and dataset dashboard together

```
visedata hyper "C:/data/hyperspectral_dataset" --label-from-parent --report
"outputs_hyper/visedata_hyper_report.html" --plot --dataset-plot --save-plot "outputs_hyper/plot_dataset.png"
```

9.6 CLI Versus Python API

The command-line interface exposes the core loading, wavelength, labelling, reporting, and dashboard workflow. More specialised HyperspectralEDA controls are intentionally left to the Python API.

Table 6. Current CLI coverage compared with the full Python API.

Capability	CLI	Python API
Load one file or directory	Yes	Yes
Supply wavelength array	Yes	Yes
Infer labels from parent folders	Yes	Yes

Capability	CLI	Python API
Generate HTML report	Yes	Yes
Generate single-cube dashboard	Yes	Yes
Generate dataset dashboard	Yes	Yes
max_cubes	No	Yes
compute_glm	No	Yes
compute_pca	No	Yes
dropout_threshold	No	Yes
recursive loading control	No	Yes
Manual NDVI/NDWI band overrides	No	Yes
plot_spectra()	No	Yes
plot_false_colour()	No	Yes
plot_ndvi()	No	Yes
plot_pca_components()	No	Yes
plot_band_stats()	No	Yes
plot_spectral_diversity()	No	Yes

Note: Advanced constructor and loading controls—including max_cubes, compute_glm, compute_pca, dropout_threshold, recursive, ndvi_nir_band, ndvi_red_band, ndwi_green_band, and ndwi_nir_band—are currently available through the Python API rather than the CLI. The specialised plotting methods are likewise Python-API features. Extend cli.py only if those controls need to be exposed at the command line.

Chapter 10 — API Reference

HyperspectralEDA.__init__()

```
HyperspectralEDA(verbose=True, wavelengths=None, max_cubes=None, compute_glm=True, compute_pca=True, dropout_threshold=0.01, ndvi_nir_band=None, ndvi_red_band=None, ndwi_green_band=None, ndwi_nir_band=None)
```

Initialise analysis configuration; does not load data.

HyperspectralEDA.load()

```
def load( self, source: Union[str, Path, List], labels: Optional[Dict[str, str]] = None, label_from_parent: bool = False, recursive: bool = True, ) -> "HyperspectralEDA":
```

Load files from a path, directory, or explicit list.

HyperspectralEDA.load_arrays()

```
def load_arrays( self, arrays: List[np.ndarray], labels: Optional[List[str]] = None, ) -> "HyperspectralEDA":
```

Load H-W-B NumPy arrays.

HyperspectralEDA.summary()

```
def summary(self) -> Dict[str, Any]:
```

Return the nested aggregate statistics dictionary.

HyperspectralEDA.get_record()

```
def get_record(self, index: int = 0) -> CubeRecord:
```

Return one CubeRecord.

HyperspectralEDA.spectral_signature()

```
def spectral_signature( self, cube_index: int = 0, row: int = 0, col: int = 0 ) -> Tuple[np.ndarray, np.ndarray]:
```

Return wavelength/index axis and one pixel spectrum.

HyperspectralEDA.compute_index()

```
def compute_index( self, cube_index: int = 0, index_name: str = "ndvi", ) -> np.ndarray:
```

Compute NDVI, NDWI, EVI, or SAVI map.

HyperspectralEDA.pca_scores()

```
def pca_scores( self, cube_index: int = 0, n_components: int = 10, ) -> Tuple[np.ndarray, np.ndarray]:
```

Return pixel-level PCA scores and explained variance.

HyperspectralEDA.plot()

```
def plot( self, cube_index: int = 0, figsize: Tuple[int, int] = (22, 20), save_path: Optional[str] = None, dpi: int = 150, rgb_bands: Optional[Tuple[int, int, int]] = None, ) -> None:
```

Render the single-cube dashboard.

HyperspectralEDA.plot_dataset()

```
def plot_dataset( self, figsize: Tuple[int, int] = (24, 24), save_path: Optional[str] = None, dpi: int = 150, ) -> None:
```

Render the dataset dashboard.

HyperspectralEDA.plot_spectra()

```
def plot_spectra( self, max_cubes: int = 20, figsize: Tuple[int, int] = (14, 6), save_path: Optional[str] = None, dpi: int = 150, ) -> None:
```

Overlay compatible mean spectra.

HyperspectralEDA.plot_false_colour()

```
def plot_false_colour( self, n: int = 12, cols: int = 4, rgb_bands: Optional[Tuple[int, int, int]] = None, figsize: Optional[Tuple] = None, save_path: Optional[str] = None, dpi: int = 150, ) -> None:
```

Render a false-colour cube grid.

HyperspectralEDA.plot_ndvi()

```
def plot_ndvi( self, n: int = 12, cols: int = 4, figsize: Optional[Tuple] = None, save_path: Optional[str] = None, dpi: int = 150, ) -> None:
```

Render NDVI maps.

HyperspectralEDA.plot_pca_components()

```
def plot_pca_components( self, cube_index: int = 0, n_components: int = 6, figsize: Optional[Tuple] = None, save_path: Optional[str] = None, dpi: int = 150, ) -> None:
```

Render PCA score images.

HyperspectralEDA.plot_band_stats()

```
def plot_band_stats( self, cube_index: int = 0, figsize: Tuple[int, int] = (16, 10), save_path: Optional[str] = None, dpi: int = 150, ) -> None:
```

Render six per-band diagnostic curves.

HyperspectralEDA.plot_spectral_diversity()

```
def plot_spectral_diversity( self, figsize: Tuple[int, int] = (10, 8), save_path: Optional[str] = None, dpi: int = 150, ) -> None:
```

Render cosine-similarity heatmap.

HyperspectralEDA.report()

```
def report(self, output_path: str = "viseda_hyperspectral_report.html") -> str:
```

Write the HTML report and return its path.

Chapter 11 — Troubleshooting

No valid cubes found

Check that the path contains supported files, optional dependencies are installed, and the arrays have two or three dimensions. Inspect verbose messages and `summary()['inventory']['corrupt_paths']`.

Ground-truth masks are analysed as cubes

Move `*_gt.mat` files outside the analysis directory or build an explicit file list containing only image cubes.

Wavelength length does not match band count

Supply a wavelength array for the specific sensor/cube or analyse different band-count groups separately.

EV1 is extremely large

Confirm reflectance scaling and wavelength mapping. Raw digital numbers can make the EVI denominator close to zero.

GLCM is blank or unavailable

Install `scikit-image`, `scikit-learn`, and `OpenCV`; set `compute_glcmm=True`. The implementation catches failures and leaves texture fields as `None`.

PCA is blank

Install `scikit-learn` and set `compute_pca=True`. Single-band cubes cannot produce a spectral PCA profile.

Spectral diversity says at least two cubes are required

At least two valid cubes must share the same dominant band count.

The .mat loader chooses the wrong array

The current loader selects the largest non-private MATLAB array. Keep ground truth and auxiliary arrays in separate files or customise `_read_cube()`.

ENVI or GeoTIFF loading fails

Install `spectral` for ENVI formats and `rasterio` for GeoTIFF. Verify that sidecar files are available.

The Word TOC is blank

Open the document in Microsoft Word, press `Ctrl+A` then `F9`, or right-click the TOC and choose `Update Field`.

The Word output is permission denied

Close the existing DOCX if it is open. The generator also creates a timestamped filename when the target is locked.

Chapter 12 — Roadmap

Version 1.0.0 — Current Documented Release

- Unified analysis for single cubes and directories.
- MATLAB, NumPy, ENVI, and GeoTIFF support.
- Per-band statistics, SNR, MAD noise, saturation, and quality checks.
- NDVI, NDWI, EVI, and SAVI maps.
- PCA variance and component images.
- GLCM texture from the first principal component.
- Dataset dashboard, spectral overlay, false colour, NDVI, PCA, band-statistics, and diversity plots.
- Self-contained HTML report and VisEDA CLI integration.

Roadmap

- Explicit sensor profiles and wavelength validation.
- Automatic exclusion of ground-truth masks and metadata arrays.
- Radiometric calibration helpers for digital numbers, radiance, and reflectance.
- Spectral resampling to a common wavelength grid.
- Bad-band recommendations based on combined SNR, noise, smoothness, and saturation.
- Additional indices and user-defined index expressions.
- Endmember extraction and spectral-angle mapping.
- Interactive dashboards and exportable per-cube tables.
- Chunked or memory-mapped processing for very large cubes.

Appendix A — Complete CubeRecord Field Reference

Field	Group	Meaning
path	Identity	Source path or array placeholder.
label	Identity	Optional class label.
file_ext	Identity	File extension or array.
file_size_kb	Identity	File size in kilobytes.
height	Shape	Spatial height.
width	Shape	Spatial width.
bands	Shape	Spectral band count.
dtype	Shape	Internal dtype.
global_mean	Global	Overall cube mean.
global_std	Global	Overall cube standard deviation.
global_min	Global	Overall minimum.
global_max	Global	Overall maximum.
dynamic_range	Global	$\text{global_max} - \text{global_min}$.
band_means	Per-band	Spatial mean for each band.
band_stds	Per-band	Spatial standard deviation for each band.
band_mins	Per-band	Spatial minimum for each band.
band_maxs	Per-band	Spatial maximum for each band.
band_snr	Per-band	Mean/std for each band.
band_noise_mad	Per-band	Median absolute deviation for each band.
band_saturation_frac	Per-band	Fraction at the cube maximum.
spectral_smoothness	Quality	Mean adjacent band-mean difference.
inter_band_corr	Quality	Mean off-diagonal band correlation.
n_dropout_bands	Quality	Low-variance band count.
snr_mean	Quality	Mean band SNR.
spatial_mean_map	Spatial map	Mean across bands.
spatial_std_map	Spatial map	Std across bands.
ndvi_mean/std	Index	NDVI map summary.
ndwi_mean/std	Index	NDWI map summary.
evi_mean/std	Index	EVI map summary.
savi_mean/std	Index	SAVI map summary.
pca_variance_ratio	PCA	Explained-variance array.
glcm_contrast	Texture	GLCM contrast.
glcm_dissimilarity	Texture	GLCM dissimilarity.
glcm_homogeneity	Texture	GLCM homogeneity.
glcm_energy	Texture	GLCM energy.
glcm_correlation	Texture	GLCM correlation.
glcm_asm	Texture	Angular second moment.
is_corrupt	Status	Whether analysis failed.

Appendix B — summary() Schema

```
{
  "inventory": {
    "total_cubes", "valid_cubes", "corrupt_cubes",
    "corrupt_paths", "band_distribution",
    "format_distribution", "label_distribution"
  },
  "spatial": {
    "height", "width", "bands", "file_size_kb"
  },
  "spectral_stats": {
    "global_mean", "global_std", "dynamic_range",
    "cross_cube_mean_spectrum",
    "cross_cube_std_spectrum",
    "dataset_mean_snr_per_band",
    "dataset_mean_noise_per_band",
    "dominant_band_count", "n_matching_cubes"
  },
  "spectral_quality": {
    "snr_mean", "spectral_smoothness",
    "inter_band_corr", "n_dropout_bands"
  },
  "spectral_indices": {
    "ndvi", "ndwi", "evi", "savi"
  },
  "texture": {
    "glcm_contrast", "glcm_dissimilarity",
    "glcm_homogeneity", "glcm_energy",
    "glcm_correlation", "glcm_asm"
  },
  "pca": {
    "mean_variance_ratio", "n_components_95pct_mean"
  },
  "labels": {
    "label_distribution", "class_imbalance_ratio"
  }
}
```

Appendix C — Glossary

Term	Definition
Band	One wavelength channel in the cube.
Cube	A three-dimensional H-W-B hyperspectral array.
Digital number	Sensor-scaled integer before physical calibration.
Radiance	Radiant energy measured by the sensor.
Reflectance	Ratio of reflected to incident radiation; common basis for vegetation indices.
SNR	Signal-to-noise ratio; here mean divided by spatial standard deviation.
MAD	Median absolute deviation, a robust dispersion measure.
Dropout band	A band with exceptionally low spatial variability.
PCA	Principal component analysis, a linear dimensionality-reduction method.
GLCM	Grey-level co-occurrence matrix used for texture analysis.
NDVI	Normalised Difference Vegetation Index.
NDWI	Normalised Difference Water Index in the green-NIR formulation used here.
EVI	Enhanced Vegetation Index.
SAVI	Soil-Adjusted Vegetation Index.
Cosine similarity	Similarity of spectral shape independent of vector magnitude.

Appendix D — Acknowledgements

VisEDA HyperspectralEDA builds on NumPy, Matplotlib, SciPy, scikit-learn, scikit-image, OpenCV, Spectral Python, Rasterio, and python-docx. Public hyperspectral datasets such as Pavia University, Indian Pines, and Salinas provide useful benchmarks for testing spatial-spectral analysis workflows.